

Hemodinks - Documentacao Tecnica

Backend, frontend, dados, Azure e fluxos principais

O Hemodinks e uma aplicacao web composta por frontend React/Vite, API ASP.NET Core/.NET 10, banco SQL Server/Azure SQL e Azure Blob Storage para arquivos. A API usa Clean Architecture pragmatica, CQRS com MediatR, validacao em pipeline, JWT Bearer, EF Core, Serilog, Swagger, Scalar, agenda com lembretes e IMemoryCache para consultas CBHPM.

URLs

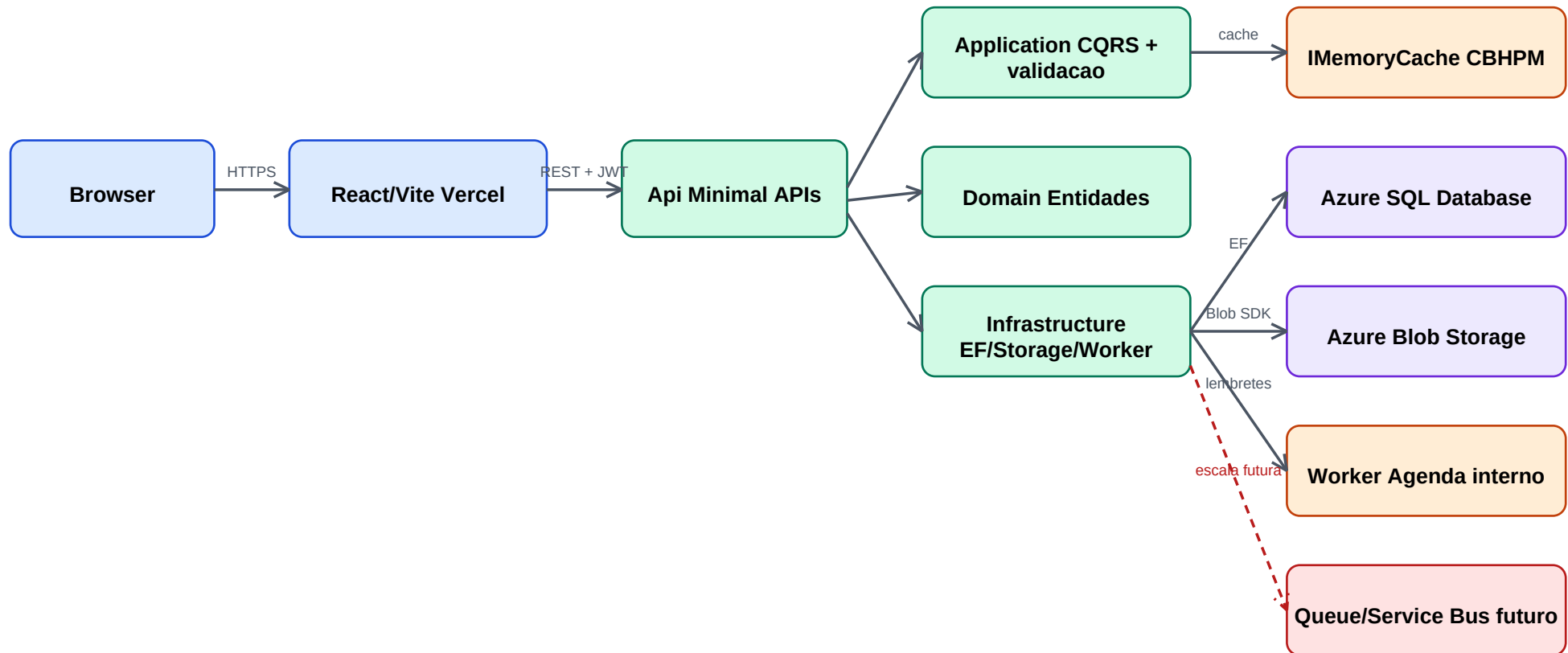
Frontend local	http://localhost:5173
Frontend producao	https://hemodinks-saude.vercel.app
API local	http://localhost:5000
Swagger	/swagger
Scalar	/scalar
OpenAPI JSON	/openapi/v1.json

Recursos Azure

- Azure SQL Database: persistencia relacional via Entity Framework Core.
- Azure Blob Storage: containers profile-photos e patient-files.
- Agenda: lembretes processados por BackgroundService interno, sem fila paga nesta fase.
- Licencas: controle de trial, plano completo e features liberadas para medicos.
- Azure Queue Storage / Service Bus: nao utilizado na versao atual; reservado para escala futura.
- IMemoryCache: cache local da API, sem recurso Azure separado.

Arquitetura e Comunicacao

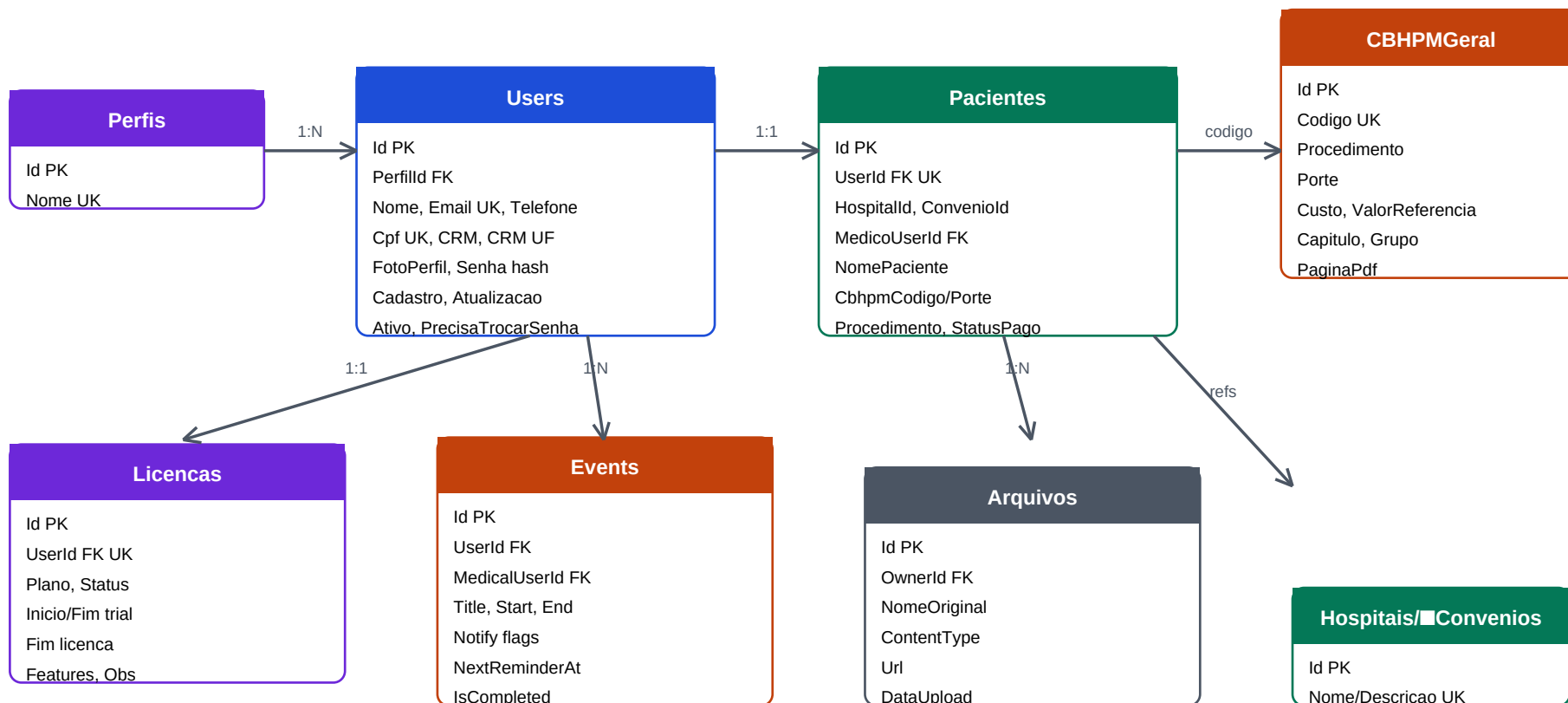
Clean Architecture pragmatica e integracoes externas



- Swagger e Scalar são servidos pela própria API e consomem o documento OpenAPI gerado por Swashbuckle.
- A consulta CBHPM aquece o cache na primeira chamada; filtros e paginação seguintes rodam em memória.
- Agenda usa **BackgroundService** interno e **NextReminderAt** para consultar apenas lembretes vencidos.
- Uploads usam **Azure Blob Storage**; o banco guarda a URL e os metadados.

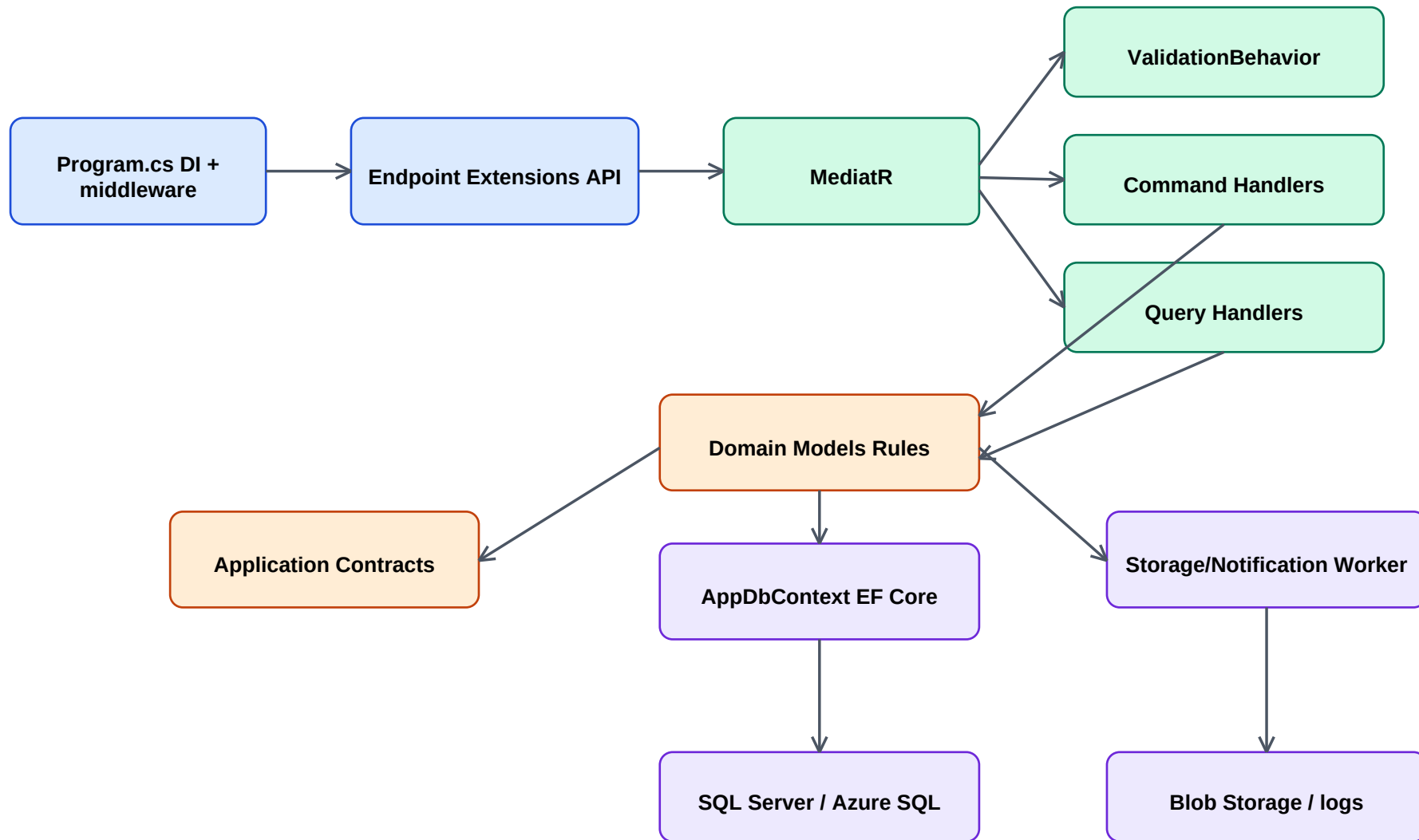
MER - Modelo Entidade Relacional

Tabelas principais, chaves e relacionamentos



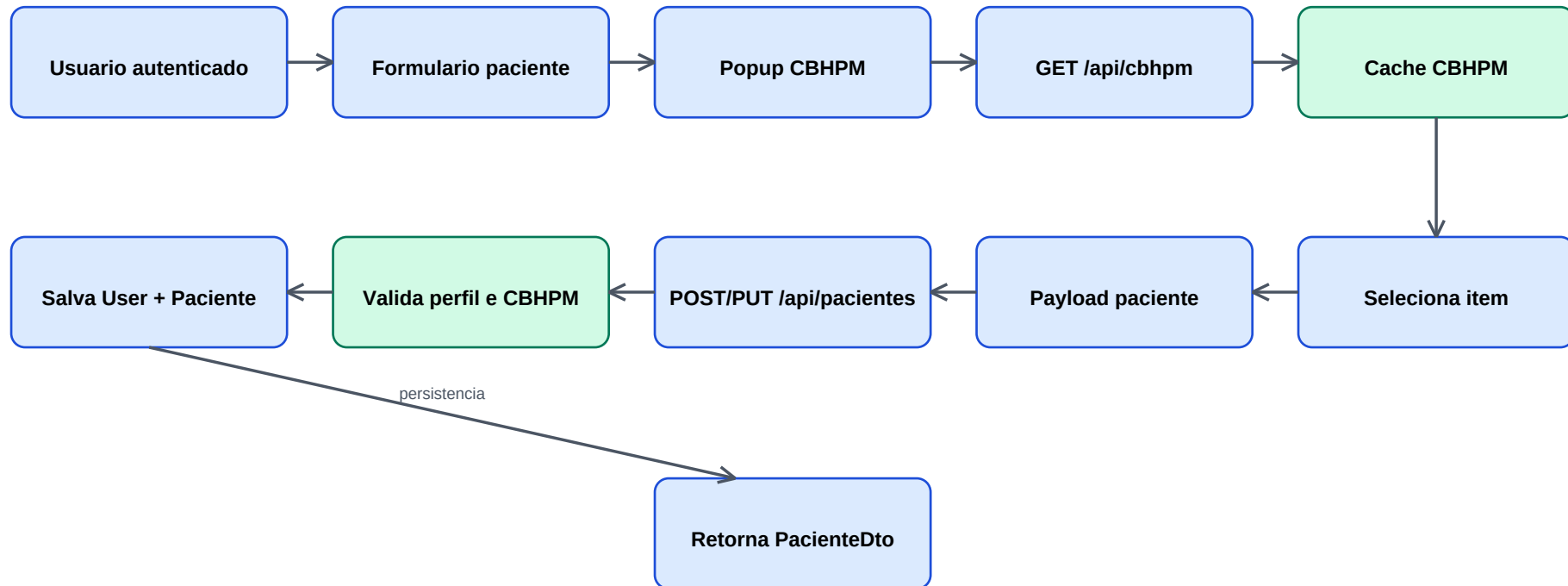
Fluxo de Classes do Backend

Minimal APIs, MediatR, validacao, contratos e Infrastructure



Fluxo de Regra de Negocio

Cadastro e edicao de paciente com selecao CBHPM

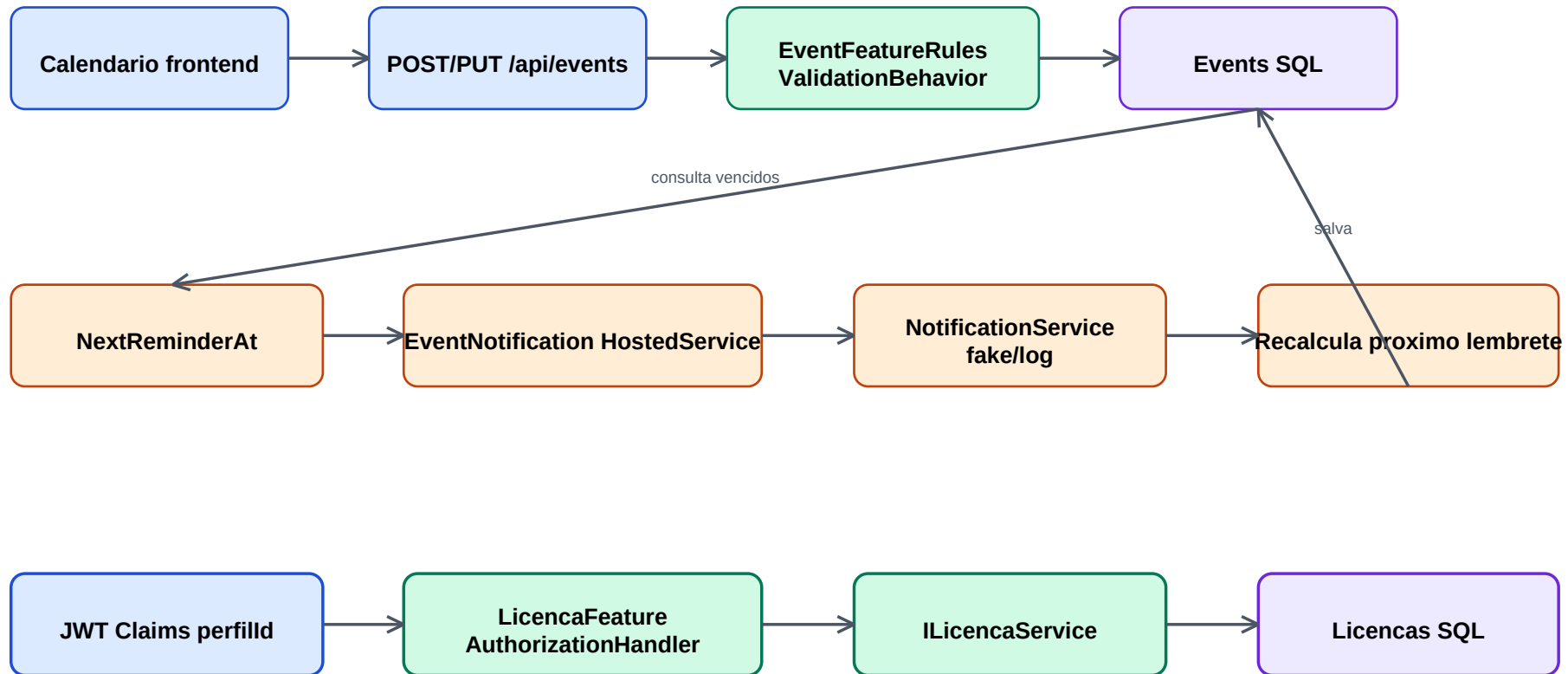


Regras principais

- Administrador pode cadastrar e editar pacientes; medico tem acesso conforme vinculo; paciente acessa o proprio cadastro.
- Se CbhpmCodigo existir, a API busca o procedimento no cache e usa os dados oficiais de codigo, porte e procedimento.
- Anexos sao enviados separadamente por endpoint multipart e armazenados no Azure Blob Storage.

Agenda, Lembretes e Licencas

Eventos, notificacoes internas e controle de acesso por feature

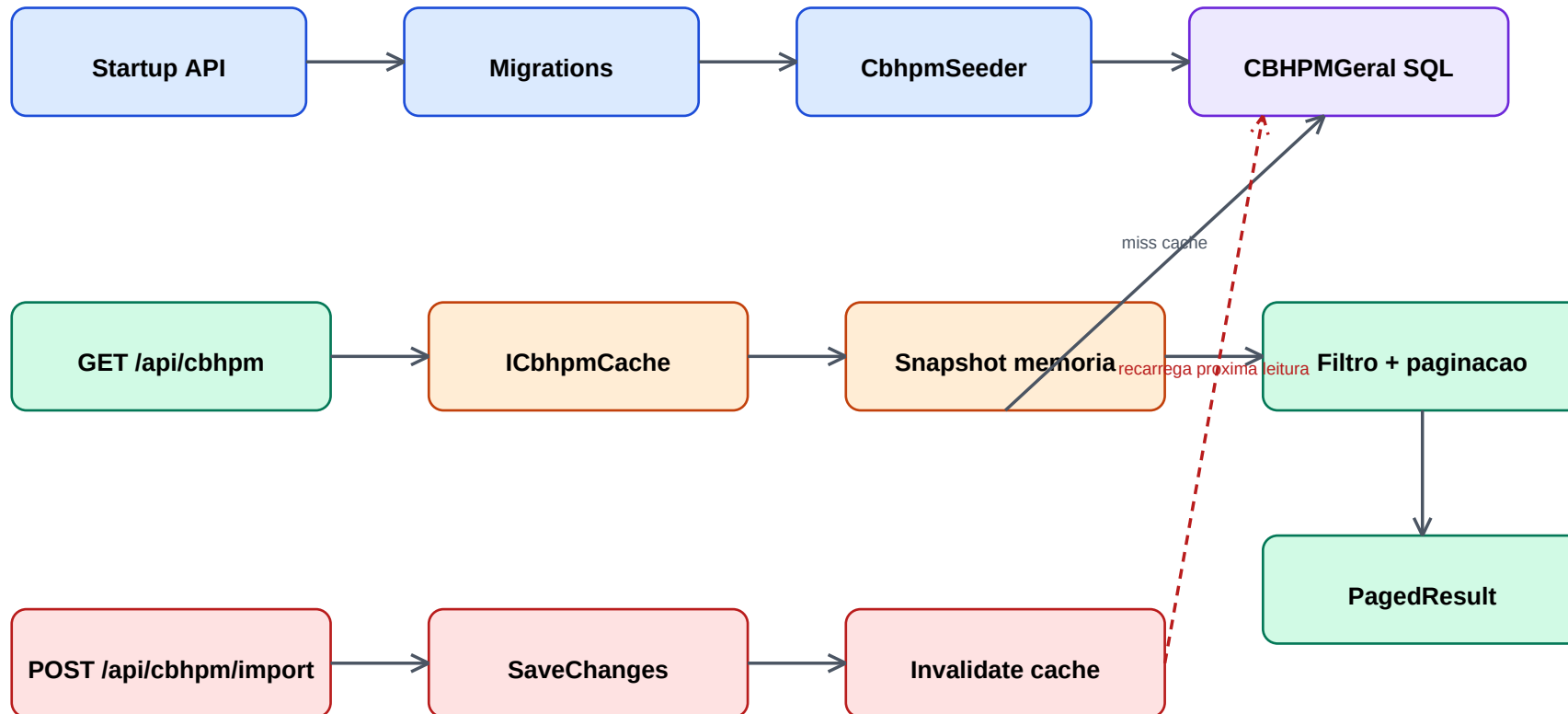


Regras principais

- Agenda pergunta se deve notificar usuario e/ou perfil medico e salva reminderPeriodMinutes.
- Worker interno consulta somente lembretes vencidos por NextReminderAt; evento concluido nao gera novos lembretes.
- Licencas controlam Dashboard, Pacientes e CBHPM por feature; administrador pode liberar plano completo.
- NotificationService atual registra logs; a interface permite trocar por push, email, FCM ou fila futura.

Fluxo CBHPM e Cache

Seed, leitura, filtro, paginacao e invalidacao



Politica

- Expiracao absoluta de 12 horas e deslizante de 2 horas.
- Cache local por processo; em multiplas instancias, cada instancia aquece seu proprio cache.
- Importacao e seed invalidam a chave cbhpm-geral:v1.

Documentacao Viva da API

Swagger, Scalar e endpoints principais

Metodo	Rota	Uso
GET	/healthz	Health check publico
POST	/api/users/authenticate	Login JWT
GET	/api/users	Usuarios paginados
GET	/api/pacientes	Pacientes paginados
POST	/api/pacientes	Cadastro de paciente
GET	/api/cbhpm	Consulta CBHPM paginada com cache
GET	/api/dashboard/summary	Resumo do dashboard
GET	/api/events	Eventos da agenda por periodo
POST	/api/events	Criacao de evento com lembretes
POST	/api/events/{id}/complete	Conclusao de evento
GET	/api/licencas/current	Licenca do usuario autenticado
PUT	/api/licencas/users/{userId}	Atualizacao de licenca admin
GET	/api/hospitais	Catalogo de hospitais
GET	/api/convenios	Catalogo de convenios

Interfaces de documentacao

- Swagger UI: /swagger
- Scalar UI: /scalar
- OpenAPI JSON usado pelo Scalar: /openapi/v1.json
- Swagger JSON: /swagger/v1/swagger.json